PlantAI

AI-Powered Plant Disease Detection System

This document provides a complete technical explanation of the PlantAI system — including the Machine Learning model, Flutter Mobile Frontend, FastAPI Backend, and the React Admin Dashboard — along with the full end-to-end workflow of the project.

System Components:

- Machine Learning: EfficientNetB0 CNN with Transfer Learning
- Mobile App: Flutter (Dart) with on-device TFLite inference
- Backend API: Python FastAPI with MongoDB (async motor driver)
- Admin Dashboard: React.js with Material-UI and Recharts

Prepared by: PlantAI Engineering Team | 2026

Section 1: Machine Learning Model & Preprocessing

1.1 Model Architecture — EfficientNetB0 (Transfer Learning)

We used Transfer Learning with Google's EfficientNetB0, a pre-trained Convolutional Neural Network (CNN). Instead of training a model from scratch (which would require millions of images and weeks of GPU time), we took a model already trained on ImageNet (14 million everyday images) and fine-tuned it to detect plant diseases.

Why EfficientNetB0?

- Compound Scaling: Unlike older models (ResNet, VGG), EfficientNet uniformly scales depth, width, and resolution together, giving maximum accuracy at minimum size.
- Mobile-Friendly: The model is ~15–20 MB, small enough to embed directly inside a mobile app without excessive battery drain.
- Pre-trained Knowledge: The base layers already knew how to detect edges, textures, and shapes from ImageNet. We only needed to teach the 'top' layers to recognize plant diseases.

Custom Top Layers Added:

- Frozen Base: All original EfficientNet layers were frozen (weights locked) so they retained ImageNet knowledge.
- Global Average Pooling: Converts the final 3D feature map into a 1D vector.
- Dropout Layer (50%): Randomly deactivates 50% of neurons during training to prevent overfitting (memorizing training data).
- Dense Softmax Output Layer: Final layer with 38 nodes (one per plant disease class). Softmax converts raw scores into probability percentages that add up to 100%.

1.2 Dataset & Data Augmentation

The model was trained on the PlantVillage dataset, a publicly available dataset containing 54,000+ leaf images across 38 disease categories covering 14 crop species including Apple, Tomato, Grape, Corn, Potato, and more.

Preprocessing Pipeline:

- Resizing: Every image was resized to exactly 224×224 pixels (EfficientNetB0's required input).
- Normalization: Pixel values (0–255) were divided by 255 to scale them to (0.0–1.0). Neural networks converge much faster with small numbers.

Data Augmentation (applied live during training):

- Rotation: Random rotations up to 40 degrees.
- Width/Height Shift: Random translation by up to 20% in any direction.
- Shear: Random angular distortion up to 20%.
- Zoom: Random zoom in/out by 20%.

• Horizontal Flip: Mirror image randomly.

This augmentation made 10,000 original images effectively become infinite variations, making the model robust to different angles, lighting, and camera distances.

1.3 Training Configuration

• Optimizer: Adam — automatically adjusts the learning rate per neuron for faster convergence.

• Loss Function: Categorical Crossentropy — the standard formula for multi-class classification.

• Early Stopping: Monitored validation accuracy; stopped automatically if no improvement for 5 epochs.

• ReduceLROnPlateau: Automatically reduced the learning rate when the model's progress plateaued.

1.4 Dual Export: .h5 & .tflite

After training, the model was exported in two formats to support both backend and mobile usage:

• .h5 (HDF5 Format): The full-size Keras model saved on the Python server for 'Cloud Inference' — used as a fallback for complex cases.

• .tflite (TensorFlow Lite): The mobile-optimized version, converted using the TFLite Converter with quantization. 32-bit floats are compressed to 8/16-bit integers, making the model 3–4x faster on mobile with near-zero accuracy loss. This file is embedded directly inside the Flutter app's assets folder.

Section 2: Flutter Mobile App (Frontend)

2.1 Technology Stack

- Language: Dart (compiled to native ARM machine code — not JavaScript)
- Framework: Flutter 3.x with Material Design 3
- State Management: Provider package (global user state via InheritedWidget)
- Typography: Google Fonts — Outfit (headings) and Plus Jakarta Sans (body)
- Key Packages: tflite_flutter, camera, image_picker, http, shared_preferences, provider, jwt_decoder, device_info_plus, flutter_local_notifications

2.2 Page Architecture (5 Screens)

Login / Register Page (login_page.dart)

Full-screen authentication wall. On successful login, the backend returns a signed JWT access_token, which is saved to SharedPreferences (phone's local key-value storage). Every subsequent API request includes this token in the HTTP Authorization: Bearer header.

Home Page (home_page.dart)

Main hub of the app. Built as a StatefulWidget listening to user state changes. Contains: user greeting card, stats bar (total scans, diseases found), a 'Recent Scans' section showing the last 5 results fetched via PredictionHistoryService.getHistory(), and a full deletable scan history list.

Scanner Page (scanner_page.dart)

The most complex widget, split into two parts: (1) The Camera View — manages the live camera preview using the camera package with an animated 4-corner bracket overlay guiding the user. (2) The Result Bottom Sheet — a DraggableScrollableSheet that slides up showing disease name, confidence pill, treatment plan, and Yes/No feedback buttons.

Library Page (library_page.dart)

A searchable, scrollable encyclopedia of all 38 plant diseases the model can detect. Shows disease info, symptoms, and treatment cards.

Notification Settings Page (notification_settings_page.dart)

Allows users to set up scheduled local push notifications using the flutter_local_notifications package with the timezone package for accurate IST scheduling.

2.3 Core Scanning Workflow (Step-by-Step)

Step 1 — Image Acquisition: User taps 'Capture.' The camera package captures a CameraImage frame from the device hardware and saves it as a temporary .jpg file using path_provider.

Step 2 — Dart-Side Preprocessing: The image package reads the raw JPEG, crops it to a square (center-crop), and resizes it to exactly 224×224 pixels — matching the model's required input.

Step 3 — TFLite Inference: The tflite_flutter Interpreter loads model.tflite from assets. The 224×224 pixels are converted into a Float32List (values between 0.0 and 1.0) and fed into the interpreter. The phone's CPU/GPU performs millions of matrix multiplications in ~200 milliseconds.

Step 4 — Post-Processing: The model outputs a List of 38 confidence scores. The app finds the highest score (argmax). If below 0.5 threshold, marks result as 'Unknown' and asks for a clearer photo. If above 0.5, maps the index to the corresponding label in labels.txt.

Step 5 — Background Logging: Simultaneously, the app silently fires an HTTP POST to /api/predict/log-local with disease code, confidence, and device metadata, saving the scan to MongoDB for the dashboard to read.

Section 3: FastAPI Python Backend

3.1 Technology Stack

- Language: Python 3.10+
- Framework: FastAPI (async, built on Starlette + Pydantic for auto validation)
- Database: MongoDB with motor async driver (non-blocking queries)
- Auth: JWT tokens signed with python-jose (HS256 algorithm), 7-day expiry
- Password Security: passlib with bcrypt hashing (industry standard, one-way hash)
- Server: Uvicorn ASGI server for high-concurrency async request handling

3.2 File & Module Structure

```
plant_disease_backend/ ■■■ app/ ■ ■■■ main.py # Entry point: registers routers, adds CORS middleware ■ ■■■ config.py # Reads .env variables (JWT secret, MongoDB URL) ■ ■■■ database/ ■ ■ ■■■ mongodb.py # MongoDB connection pool on startup ■ ■ ■■■ models.py # Pydantic response models ■ ■■■ api/ ■ ■ ■■■ auth.py # /api/auth — Login, Register, token verification ■ ■ ■■■ predictions.py # /api/predict — Log scans, feedback, history ■ ■ ■■■ admin.py # /api/admin — Dashboard stats and analytics ■ ■■■ services/ ■ ■■■ ml_service.py # Loads .h5 Keras model for cloud inference ■ ■■■ logging_service.py # Saves predictions & feedback to MongoDB ■ ■■■ recommendation_service.py # Returns treatment text per disease
```

3.3 API Endpoints — Detailed

POST /api/auth/register

Receives {name, email, password}. Checks if email exists. Runs bcrypt.hash(password) — a one-way mathematical operation. Stores the hash (never the plain password). Returns a signed JWT access_token.

POST /api/auth/login

Receives {email, password}. Fetches user document from MongoDB. Runs bcrypt.verify(plain, hash) to check the password. On success, creates and returns a new signed JWT with 7-day expiry.

POST /api/predict/log-local (Most important endpoint)

Called silently by the Flutter app after every scan. Receives {disease_code, confidence, image_name, processing_time_ms}. The LoggingService creates a new document with UTC timestamp and saves it to the predictions collection in MongoDB. This is what the dashboard's Recent Scans table reads.

POST /api/predict/feedback

Receives {prediction_id, was_correct, comments} as JSON. Saves a new document to the feedback collection. This drives the entire System Accuracy calculation on the dashboard.

GET /api/admin/stats?days=7

Polled by the React Dashboard every 5 seconds. Runs a MongoDB Aggregation Pipeline counting total predictions, splitting by local_inference flag (Cloud vs Edge), calculating average confidence, and counting unique users. A second aggregation reads the feedback collection to compute the Bayesian-smoothed accuracy percentage.

GET /api/admin/model-metrics

Calculates Accuracy, Precision, Recall, and F1 Score from the feedback collection. Uses Bayesian Smoothing (prior: 50 samples at 96% accuracy) so metrics always look realistic. Also builds historical accuracy trend data for the chart, grouped by date.

GET /api/admin/analytics/daily?days=7

Drives the bar charts on the dashboard. Converts UTC timestamps to IST (UTC+5:30) by adding 330 minutes before grouping. Returns a filled array for every single day — even days with 0 scans get a 0 entry so the chart has no gaps.

Section 4: React Admin Dashboard

4.1 Technology Stack

- Framework: React 18 with functional components and hooks
- UI Library: Material-UI (MUI) v6 for cards, tables, buttons, chips
- Charts: Recharts — LineChart, AreaChart, BarChart, PieChart
- Animations: Framer Motion for page entrance and card slide-in effects
- HTTP Client: Axios with a base URL configured to http://127.0.0.1:8000
- Routing: React Router v6 for SPA (Single Page Application) navigation
- State: useState + useEffect hooks (no Redux needed — lightweight app)
- Themes: Custom ThemeContext for Dark/Light mode + multi-language translations (English, Hindi, Marathi)

4.2 Page Architecture (7 Pages)

Login Page (Login.js)

Full-screen admin authentication wall with email/password form. Calls POST /api/auth/login and stores the JWT token in localStorage. Protects all other routes — unauthenticated users are redirected here automatically.

Dashboard Page (Dashboard.js) — The Main Hub

The most important page. Uses setInterval to poll the backend every 5 seconds and refresh all data automatically. Displays 5 live stats cards: Total Scans, System Accuracy (Bayesian-smoothed from real feedback), Model Confidence (average of all scan confidence scores), Edge Inference Count, and Unique Users. Also shows: Disease Frequency Bar Chart, Hourly Activity Line Chart, Daily Scans Area Chart, and the Recent Scans Table with the last 10 predictions from MongoDB.

Disease Frequency Analysis (Analytics.js)

Shows which diseases are being detected most often. Fetches the top_diseases array from /api/admin/stats and renders it as a horizontal bar chart and a ranked table. Each disease shows its count and average confidence score.

Model Accuracy Tracking (ModelMonitoring.js)

Fetches /api/admin/model-metrics. Displays 4 live KPI cards: Accuracy %, Precision %, Recall %, and F1 Score %. Shows a historical Accuracy Trend Line Chart over time, built from daily feedback aggregations. Also shows a static Model Version History table with rollback options for administrators.

Dataset Update Monitoring (DatasetManagement.js)

Shows total scan count, unique disease classes detected, total feedback count, and unique contributing users. Includes a Field Scan Volume area chart and the full Recent Field Scans table showing every scan

logged from the Flutter app with date, disease, confidence, inference mode, and user ID.

Feedback Page (Feedback.js)

Fetches /api/admin/feedback. Shows a Sentiment Pie Chart with Correct vs Incorrect split. Displays summary cards with the exact count of correct and incorrect predictions. Below that, a full paginated table shows every individual feedback entry with its date, prediction ID, correctness badge, actual disease (if user specified), and any comment text.

Settings Page (Settings.js)

Allows administrators to change the UI language (English/Hindi/Marathi), toggle Dark/Light mode, change the auto-refresh polling interval, and update their admin profile.

4.3 The Live Data Architecture

The dashboard uses React's useEffect hook with a cleanup function to implement real-time polling:

```
useEffect(() => { fetchDashboardData(); // Fetch immediately on mount const interval =
setInterval(() => { fetchDashboardData(); // Re-fetch every 5 seconds }, 5000); return
() => clearInterval(interval); // Cleanup on unmount }, []);
```

This means the dashboard is always showing data that is at most 5 seconds old. The moment a user scans a leaf in the Flutter app, the scan is saved to MongoDB, and within the next 5-second poll cycle the dashboard will automatically update all charts and the Recent Scans table without the administrator needing to refresh the browser.

4.4 Multi-Language Support

The entire dashboard UI is internationalized. A custom ThemeContext provides a translation function t('key'). All visible text strings are stored in three language objects inside ThemeContext.js — English, Hindi, and Marathi. Switching language from the Settings page instantly re-renders all text across all 7 pages without any page reload.

Section 5: Complete End-to-End Project Workflow

This section describes the complete journey of data from the moment a farmer points their phone at a sick leaf to the moment the administrator sees the updated live statistics on their laptop dashboard.

Phase 1: User Authentication

- The farmer opens the PlantAI Flutter app on their Android phone.
- They register/login on the Login Page. The app sends credentials to POST /api/auth/login on the FastAPI backend.
- The backend verifies the bcrypt password hash and returns a signed JWT token.
- The JWT token is saved to SharedPreferences on the phone. From this moment, every API call is authenticated.

Phase 2: Leaf Scanning (Edge AI — Offline)

- The farmer navigates to the Scanner Page. The phone's rear camera opens with a live preview.
- An animated 4-corner bracket overlay guides the farmer to center the leaf.
- The farmer taps 'Capture.' The camera takes a high-resolution photo.
- Dart preprocessing: The image package crops the photo to a square and resizes it to 224×224 pixels.
- The tflite_flutter Interpreter loads model.tflite from the app's assets.
- The 224×224 pixel array is fed into the TFLite model. 38 confidence scores are returned.
- The app finds the highest score. If above 50% threshold, it maps the index to the disease label. If below, it asks the user to try again with a clearer photo.
- The entire inference process takes approximately 150–300 milliseconds. No internet required.

Phase 3: Results Display & Logging

- The Result Bottom Sheet slides up from the bottom of the screen.
- It shows: the leaf photo, disease name, plant type, confidence percentage, a Healthy/Diseased badge, and a full treatment recommendation.
- SIMULTANEOUSLY in the background: The app fires an HTTP POST to /api/predict/log-local with the disease code, confidence, image name, and device metadata.
- The FastAPI backend receives this, creates a MongoDB document with a UTC timestamp, and saves it to the predictions collection.

Phase 4: User Feedback Submission

- At the bottom of the result screen, the farmer sees: 'Was this prediction accurate?'
- They tap 'Yes' (correct) or 'No' (incorrect). Optionally they can add a comment.
- The app sends this to POST /api/predict/feedback with the prediction_id and was_correct flag.

• The backend saves a new document to the feedback collection in MongoDB.

Phase 5: Dashboard Auto-Update

• The React Admin Dashboard is open on the administrator's laptop browser.

• Every 5 seconds, the Dashboard page calls GET /api/admin/stats.

• The backend runs MongoDB aggregation pipelines: counting total scans, computing the smoothed accuracy from feedback, grouping by date for charts.

• The dashboard receives the fresh JSON data and React automatically re-renders all the charts, KPI cards, and the Recent Scans table.

• The administrator sees the new scan appear in the 'Recent Field Scans' table in real-time.

• The System Accuracy and Model Confidence cards update to reflect the new data point.

Phase 6: Admin Response (Model Monitoring)

• If the administrator notices accuracy dropping (many 'No' votes from farmers), they visit the Model Accuracy Tracking page.

• They can see the day-by-day accuracy trend from the historical feedback chart.

• They can trigger the 'Retrain Model' button, which would kick off a new training job.

• After retraining, the new model replaces the old .h5 on the server and the .tflite inside the Flutter app.

Architecture Summary

Component	Technology	Role
ML Model	EfficientNetB0 + TFLite	Detect plant disease from leaf image
Mobile App	Flutter + Dart	Camera, on-device inference, user UI
Backend API	Python FastAPI + MongoDB	Auth, logging, analytics aggregation
Admin Dashboard	React + MUI + Recharts	Live monitoring, metrics, feedback view
Database	MongoDB (motor async)	Store predictions, users, feedback
Auth System	JWT + bcrypt	Secure login, token-based session management

---